

Aprendizado por Reforço Tabular no Controle de Semáforos e na Navegação de Agentes em uma Cidade Simplificada

Henrique Kops¹, [demais autores]¹

¹Escola Politécnica, Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS)
Porto Alegre, RS, Brasil

[e-mails dos autores]

Abstract. *This work applies tabular reinforcement learning to a simplified urban mobility problem in which agents, representing vehicles, cross a street grid with one obstacle and traffic lights until they reach assigned destinations. Two kinds of learners are trained independently, the agents that choose directions and the traffic lights that choose which axis receives the green signal. Both use Q-learning with an epsilon-greedy policy. The formulation is inspired by previous studies on multiagent traffic signal control and on mobile robot navigation in grid environments. Runs with different hyperparameters are compared in terms of average travel time. The results indicate [summary of the results].*

Resumo. *Este trabalho aplica o aprendizado por reforço tabular a um problema simplificado de mobilidade urbana, no qual agentes que representam veículos percorrem uma grade de ruas com um obstáculo e semáforos até destinos previamente definidos. Dois tipos de aprendizes são treinados de forma independente, os agentes que escolhem direções e os semáforos que escolhem qual eixo recebe o sinal verde. Ambos utilizam Q-learning com política ϵ -greedy. A formulação se inspira em estudos sobre controle semaforico multiagente e sobre navegação de robôs em ambientes em grade. Execuções com diferentes hiperparâmetros são comparadas quanto ao tempo médio de viagem. Os resultados indicam [síntese dos resultados].*

1. Introdução

O congestionamento urbano gera perda de tempo, consumo de combustível e emissão de poluentes, e uma das medidas de mitigação de menor custo é o controle inteligente dos semáforos [Jin et al. 2011]. Os controladores tradicionais operam com tempos fixos, definidos previamente a partir de médias de fluxo, e por isso não se adaptam às variações do tráfego ao longo do dia. O aprendizado por reforço oferece uma alternativa interessante, pois permite que um agente aprenda uma política de decisão apenas pela interação com o ambiente, sem exigir um modelo prévio do comportamento do tráfego.

O problema abordado neste trabalho é o de uma cidade simplificada, representada por uma grade com seis ruas, quatro quadras, nove cruzamentos controlados por semáforos e um buraco em uma das vias. Agentes que representam veículos entram pela borda do mapa, cada um com um destino, e devem chegar a ele no menor tempo possível, evitando o buraco e respeitando os sinais. Dois tipos de aprendizes atuam simultaneamente, os próprios agentes, que escolhem a direção de cada passo, e os semáforos, que escolhem qual eixo recebe o verde. O cenário é pequeno o bastante para ser tratado com Q-learning tabular e, ao mesmo tempo, rico o suficiente para exibir efeitos da interação entre os aprendizes.

A literatura já demonstrou a aplicabilidade do aprendizado por reforço ao controle semafórico em redes com vários cruzamentos [Jin et al. 2011] e à navegação de robôs em ambientes em grade com obstáculos [Hafez e Loo 2015], além de ter investigado a redução da dimensão do espaço de estados para acelerar a convergência [Curran et al. 2015]. Este trabalho se baseia nesses estudos, sem reproduzi-los, e reúne as duas aplicações em um único ambiente.

As contribuições são a formulação do problema como dois processos de decisão de Markov que interagem entre si, uma implementação em Python com interface gráfica que permite treinar, observar e inspecionar as políticas aprendidas, e uma comparação controlada de execuções com diferentes hiperparâmetros. O restante do artigo apresenta a fundamentação teórica na Seção 2, os trabalhos relacionados na Seção 3, a metodologia na Seção 4, os resultados na Seção 5, a discussão na Seção 6 e a conclusão na Seção 7.

2. Fundamentação Teórica

No aprendizado por reforço, um agente interage com um ambiente em instantes discretos de tempo. Em cada instante o agente observa um estado s , escolhe uma ação a e recebe do ambiente uma recompensa r e o novo estado s' . O problema é formalizado como um processo de decisão de Markov, definido pelos conjuntos de estados e de ações, pela dinâmica de transição e pela função de recompensa [Sutton e Barto 2018]. O objetivo do agente é encontrar uma política π , que associa a cada estado uma ação, capaz de maximizar o retorno esperado, dado por

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (1)$$

em que $\gamma \in [0, 1)$ é o fator de desconto, que controla o peso atribuído às recompensas futuras. Valores próximos de zero tornam o agente imediatista, e valores próximos de um fazem com que ele considere consequências de longo prazo.

O Q-learning [Watkins e Dayan 1992] é um algoritmo de diferença temporal que não exige um modelo do ambiente e, por isso, é classificado como livre de modelo. Ele

estima a função valor de ação $Q(s, a)$, que representa o retorno esperado ao escolher a ação a no estado s e seguir a política ótima depois disso. Em problemas com poucos estados e ações, essa função é armazenada em uma tabela, e após cada passo o valor correspondente é atualizado por

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2)$$

em que α é a taxa de aprendizado. O termo entre colchetes é chamado de erro de diferença temporal, pois compara o que foi observado, a recompensa recebida somada ao melhor valor estimado do estado seguinte, com a estimativa anterior. Uma taxa α pequena faz com que cada valor seja uma média de muitas experiências, o que é desejável quando o retorno é ruidoso, enquanto valores altos tornam o aprendizado rápido, porém instável. O algoritmo é dito fora da política porque o máximo da expressão considera a melhor ação seguinte, mesmo que o agente tenha escolhido outra ao explorar.

Para aprender, o agente precisa equilibrar a exploração de ações ainda pouco testadas e o aproveitamento do que já sabe. Uma estratégia simples e amplamente usada é a política ε -greedy, na qual, com probabilidade ε , uma ação é escolhida ao acaso e, com probabilidade $1 - \varepsilon$, escolhe-se a ação de maior valor. Costuma-se iniciar com um ε maior e reduzi-lo ao longo do treinamento, de modo que o agente explore no início e aproveite no final. Quando as recompensas são negativas e a tabela é inicializada com zeros, as ações ainda não testadas parecem melhores do que as já experimentadas, o que induz uma exploração adicional conhecida como inicialização otimista. A política aprendida é obtida de forma gulosa em relação à tabela, ou seja,

$$\pi(s) = \arg \max_a Q(s, a) \quad (3)$$

e é essa política, sem exploração, que é usada para avaliar e visualizar o comportamento final. Os hiperparâmetros que governam o aprendizado são, portanto, a taxa de aprendizado α , o fator de desconto γ e a taxa de exploração inicial ε , além de parâmetros próprios do problema, como o período de decisão dos semáforos descrito adiante.

3. Trabalhos Relacionados

Três artigos serviram de base para este trabalho. A Tabela 1 identifica cada um deles, e os parágrafos seguintes resumem suas ideias e resultados e discutem seus limites.

Jin et al. [Jin et al. 2011] propõem um controle semafórico baseado em aprendizado por reforço multiagente para redes com vários cruzamentos. O estado agrega o número de veículos em todos os cruzamentos, a ação consiste em escolher qual direção recebe o verde após cada intervalo mínimo de fase, e o objetivo é fazer com que todos os

Tabela 1. Artigos estudados

Artigo	Veículo e acesso	Ano	Qualis
Jin, Zhu e Song. Implementing traffic signal optimal control by multiagent reinforcement learning	Conferência ICCSNT, IEEE. [link]	2011	[Qualis]
Hafez e Loo. Topological Q-learning with internally guided exploration for mobile robot navigation	Neural Computing and Applications. doi.org/10.1007/s00521-015-1861-8	2015	[Qualis]
Curran, Brys, Taylor e Smart. Using PCA to efficiently represent state spaces	Preprint arXiv. arxiv.org/abs/1505.00322	2015	[Qualis]

veículos deixem a rede no menor tempo. O aprendizado usa Q-learning, com uma etapa adicional que atualiza novamente os valores ao longo da trajetória percorrida. Em um simulador com seis cruzamentos, a comparação com o controle de tempo fixo em trinta situações distintas de tráfego indicou uma redução de 20 a 30 por cento no tempo necessário para esvaziar a rede. Como limitações, o estado conjunto cresce exponencialmente com o número de cruzamentos, a rede avaliada é pequena e o desempenho é medido por um único critério, apenas em simulação.

Hafez e Loo [Hafez e Loo 2015] tratam da navegação de robôs móveis em ambientes desconhecidos e propõem o Q-learning topológico. A ideia é construir de forma incremental um mapa topológico do ambiente e usá-lo para propagar uma atualização de valor aos estados vizinhos já visitados, em vez de atualizar apenas o estado corrente, e para guiar a exploração por meio de uma recompensa interna, que mede o quanto a política aprendida mudou. O método foi comparado ao Q-learning tradicional e a um algoritmo de zonas de influência em grades de 20 por 20 células, estáticas e dinâmicas, nas quais a ação que leva a um obstáculo recebe recompensa negativa e a que leva ao objetivo recebe recompensa positiva. No ambiente estático, o Q-learning tradicional necessitou em média de 190 passos para alcançar o objetivo, enquanto o método proposto encontrou uma política ótima de 24 passos após apenas dois ciclos de aprendizado. Em ambientes que mudam durante o treinamento, o método também se adaptou mais rapidamente, ao custo de um maior número de atualizações e de um maior tempo de execução. As avaliações envolvem um único robô em grades simuladas, e o benefício em cenários com vários aprendizes permanece uma questão em aberto.

Curran et al. [Curran et al. 2015] estudam como reduzir o espaço de estados por meio da análise de componentes principais, calculada a partir de trajetórias de demonstração, de modo que o Q-learning opere em uma representação de menor dimensão. No domínio de um jogo de plataforma, aprender em apenas quatro dimensões, em vez de

nove, superou o desempenho obtido no espaço completo e convergiu mais rapidamente, enquanto representações com menos de quatro dimensões convergiram depressa para políticas ruins. Os autores destacam o compromisso entre velocidade de convergência e desempenho, já que a projeção descarta informação de baixa variância. Trata-se de um trabalho preliminar, avaliado em um único domínio.

Em conjunto, os três estudos indicam que o Q-learning tabular é aplicável tanto ao controle semafórico quanto à navegação em grade, mas eles tratam os dois problemas separadamente, avaliam em geral um único aprendiz e nenhum deles tem como foco a análise controlada de hiperparâmetros. A lacuna explorada aqui é a interação entre agentes que navegam e semáforos que aprendem no mesmo ambiente, acompanhada de uma comparação controlada de hiperparâmetros.

4. Metodologia

4.1. Ambiente

O ambiente é uma grade de 11 por 11 células, na qual três ruas horizontais e três verticais, separadas por quadras de quatro células de largura, formam nove cruzamentos e delimitam quatro quadras de casas. Cada cruzamento possui um semáforo. Quatro casas, uma em cada quadra e identificadas pelas letras A a D, são os destinos, e a célula de rua vizinha a cada casa é a sua porta. Um agente chega ao destino ao alcançar a porta correspondente e, nesse momento, deixa a simulação. A cada início do programa, um único buraco é sorteado em uma célula de rua que não seja um cruzamento nem uma porta de destino. O buraco é intransponível, e verificou-se que, em qualquer uma de suas 44 posições possíveis, todas as entradas continuam conectadas a todos os destinos.

O tempo é discreto e a unidade é o tick, no qual cada agente tenta se mover uma célula. Cada época é um cenário em que um número fixo de agentes entra por células aleatórias da borda, em instantes aleatórios dos primeiros 100 ticks, com destinos sorteados e respeitando um limite de 30 agentes simultâneos no mapa. A época termina quando todos chegam ou quando atinge 1500 ticks. A semente do cenário depende apenas do número da época, de modo que épocas de mesmo número são idênticas em execuções diferentes, o que torna as comparações pareadas. Um agente que tenta entrar em um cruzamento com sinal vermelho permanece onde está, e um que tenta entrar no buraco também permanece e é penalizado.

Optou-se por não modelar colisões entre agentes, que podem compartilhar uma mesma célula. Em uma versão preliminar, na qual cada célula comportava um único agente, agentes em sentidos opostos em ruas de largura de uma célula bloqueavam-se mutuamente. Nesses testes iniciais, em média entre 7 e 10 dos 20 agentes de cada época

chegavam ao destino e todas as épocas atingiam o limite de 1500 ticks, o que inviabilizava o aprendizado. Essa observação foi feita em poucas execuções e com outras alterações simultâneas, portanto serve apenas como justificativa de projeto.

4.2. Agentes de navegação

Cada agente de navegação é modelado como um processo de decisão de Markov no qual o estado é a célula em que ele se encontra e a ação é uma das quatro direções que levam a uma célula de rua. Existe uma tabela Q para cada destino, compartilhada por todos os agentes que se dirigem a ele, de modo que esses agentes aprendem em conjunto. A decisão ocorre a cada tick. A recompensa de cada tick combina três parcelas, conforme

$$r = -1 + 0,5 \Delta d - 10 b \quad (4)$$

em que Δd é a variação da distância até a porta, medida em número de linhas e colunas e positiva quando o agente se aproxima, e b vale um quando o agente tenta entrar no buraco e zero caso contrário. O custo de -1 por tick incentiva trajetos curtos, a parcela de distância fornece um sinal de direção a cada passo e a penalidade de -10 desencoraja o buraco. A parcela de distância é uma forma de modelagem de recompensa e, quando simétrica como aqui, não altera a política ótima na formulação clássica de Ng et al. [Ng et al. 1999], embora a igualdade exata dependa de $\gamma = 1$. Ao chegar à porta, a viagem termina e o valor alvo é apenas a recompensa recebida. O aprendizado usa a atualização da Equação 2 com política ϵ -greedy.

Essa formulação é próxima à de Hafez e Loo [Hafez e Loo 2015], que também treinam um agente em uma grade com obstáculos, usam o Q-learning com ϵ -greedy como linha de base e penalizam a ação que leva a um obstáculo. A diferença é que os autores atribuem recompensa positiva ao alcançar o objetivo, enquanto aqui se adotou um custo por passo combinado à modelagem por distância. As técnicas mais sofisticadas daquele artigo, o mapa topológico e a exploração guiada, não foram implementadas.

4.3. Semáforos

Cada um dos nove semáforos é um segundo tipo de aprendiz. Seu estado é formado pelo número de agentes nos dois braços norte e sul, pelo número de agentes nos dois braços leste e oeste, com cada contagem limitada a quatro para manter a tabela pequena, e pelo sinal atual. A ação é escolher qual eixo recebe o verde. A decisão é tomada a cada três ticks, intervalo que desempenha o papel do tempo mínimo de fase do trabalho de Jin et al. [Jin et al. 2011], e a troca de eixo custa um tick com todos os sinais vermelhos, o que desencoraja trocas frequentes. A recompensa de cada decisão é o negativo do número médio, por tick, de agentes retidos pelo sinal vermelho daquele cruzamento no período seguinte.

Os nove semáforos compartilham uma mesma tabela Q e aprendem com a atualização da Equação 2, também com política ε -greedy.

Há ainda uma regra fixa, que não é aprendida, segundo a qual o cruzamento sem agentes em seus braços permanece com todos os sinais vermelhos, pois nenhuma ação beneficiaria alguém. Essa regra incorpora conhecimento do domínio e deve ser considerada na interpretação dos resultados. Em comparação com Jin et al., que utilizam um estado conjunto de todos os cruzamentos e um objetivo de esvaziamento da rede, aqui o estado é local a cada cruzamento, o que evita o crescimento exponencial da tabela, e a recompensa se baseia na espera. Também não foi feita, neste trabalho, a comparação com o controle de tempo fixo, que é central no artigo original.

4.4. Política aprendida e hiperparâmetros

A política de cada tipo de aprendiz é a gulosa em relação à sua tabela, conforme a Equação 3. Para os agentes, ela indica, para cada célula e destino, a direção de maior valor. Para os semáforos, indica, para cada combinação de contagens e sinal, o eixo de maior valor, exceto nos cruzamentos sem demanda, regidos pela regra de fechamento. As tabelas começam zeradas, e a exploração decresce linearmente a cada época, a partir do valor inicial de ε até o mínimo de 0,02, que é atingido aos 70 por cento das épocas. A Tabela 2 reúne os hiperparâmetros e os valores de referência, escolhidos de forma heurística e não otimizados para este ambiente.

Tabela 2. Hiperparâmetros e valores de referência

Hiperparâmetro	Agente de navegação	Semáforo
Taxa de aprendizado α	0,1	0,1
Fator de desconto γ	0,99 por tick	0,9 por decisão
Exploração inicial ε	0,05	0,3
Exploração mínima ε	0,02	0,02
Período de decisão	a cada tick	3 ticks

4.5. Procedimento experimental

A avaliação utiliza o tempo médio de viagem dos agentes de cada época, em ticks, o tempo médio de espera, o número de tentativas de entrada no buraco e o tempo de processamento da época. Foram realizadas sete execuções, todas na mesma sessão do programa e, portanto, com o mesmo buraco, com o mesmo número de agentes por época e com o mesmo número de épocas, a saber **[número de agentes por época]** agentes e **[número de épocas]** épocas. A primeira é a execução de referência, com os valores da Tabela 2. As três seguintes alteram, uma de cada vez, os hiperparâmetros dos agentes de navegação, com taxa de aprendizado de 0,5, fator de desconto de 0,9 e ε inicial de 0,3. As três últimas alteram, uma por vez, os hiperparâmetros dos semáforos, com período de decisão de 8 ticks,

fator de desconto de 0,5 e ε inicial de 0,05. Separar os dois grupos permite atribuir as diferenças observadas a um único tipo de aprendiz. A implementação foi feita em Python com a biblioteca NumPy e interface gráfica em Tkinter, e o código está disponível em [\[endereço do repositório\]](#).

5. Resultados

A Figura 1 mostra a evolução do tempo médio de viagem ao longo das épocas para a execução de referência e para as três variações dos hiperparâmetros dos agentes de navegação, com suavização por média móvel de dez épocas. A Figura 2 apresenta o mesmo resultado para as variações dos hiperparâmetros dos semáforos. A Tabela 3 resume o desempenho nas dez últimas épocas de cada execução. **[descrição dos resultados após as execuções]**

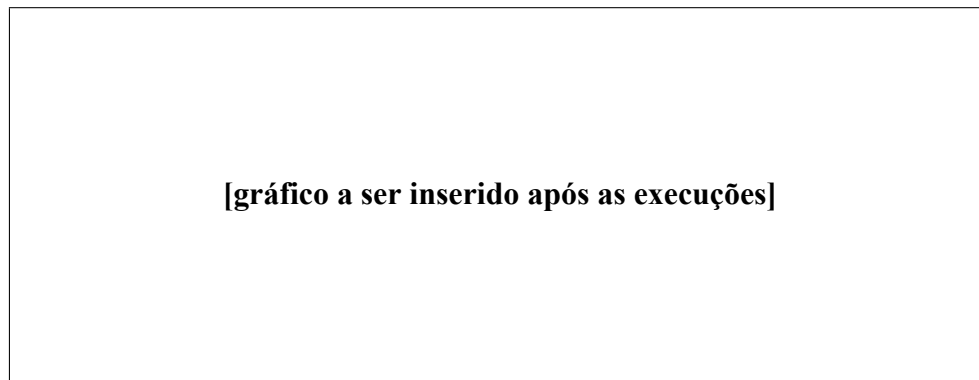


Figura 1. Tempo médio de viagem por época para a execução de referência e para as variações dos hiperparâmetros dos agentes de navegação

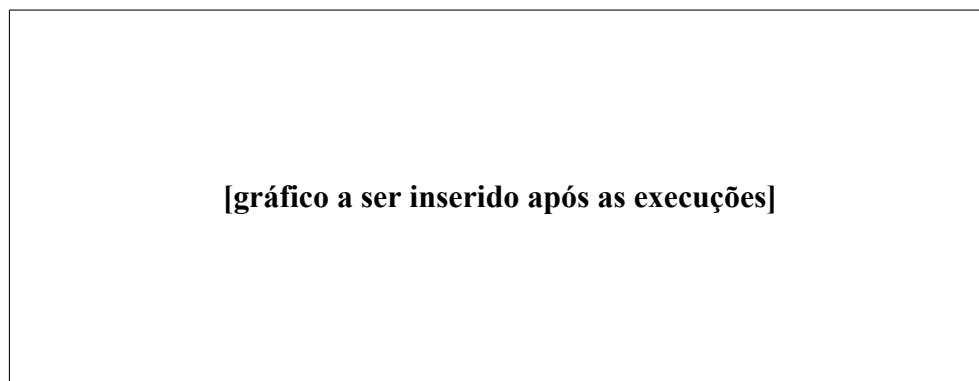


Figura 2. Tempo médio de viagem por época para a execução de referência e para as variações dos hiperparâmetros dos semáforos

6. Discussão

[interpretação dos resultados após as execuções, relacionando cada variação aos efeitos esperados da taxa de aprendizado, do fator de desconto, da exploração e do período de decisão]

Tabela 3. Desempenho médio nas dez últimas épocas de cada execução

Execução	Alteração	Viagem (ticks)	Espera (ticks)	Buraco	Tempo (s)
1	Referência	[xx]	[xx]	[xx]	[xx]
2	Agente, $\alpha = 0,5$	[xx]	[xx]	[xx]	[xx]
3	Agente, $\gamma = 0,9$	[xx]	[xx]	[xx]	[xx]
4	Agente, $\varepsilon = 0,3$	[xx]	[xx]	[xx]	[xx]
5	Semáforo, 8 ticks por decisão	[xx]	[xx]	[xx]	[xx]
6	Semáforo, $\gamma = 0,5$	[xx]	[xx]	[xx]	[xx]
7	Semáforo, $\varepsilon = 0,05$	[xx]	[xx]	[xx]	[xx]

Algumas limitações devem ser consideradas na leitura dos resultados. Do ponto de vista de cada aprendiz, o ambiente não é estacionário, pois os demais aprendizes também alteram seu comportamento ao longo do treinamento, de modo que as garantias de convergência do Q-learning, válidas para ambientes estacionários, não se aplicam diretamente. Cada configuração foi executada uma única vez com sementes fixas de aprendizado, de modo que diferenças pequenas devem ser tratadas como indícios, e não como conclusões. Os valores de referência dos hiperparâmetros não foram otimizados, e o buraco é sorteado a cada início do programa, o que impede comparar execuções de sessões distintas. Por fim, a regra que fecha os semáforos sem demanda é conhecimento do domínio e não resultado do aprendizado.

Os trabalhos revisados relatam ganhos em relação a linhas de base, o controle de tempo fixo no caso de Jin et al. [Jin et al. 2011] e o Q-learning tradicional no caso de Hafez e Loo [Hafez e Loo 2015], e nenhuma dessas comparações foi realizada aqui. Os resultados apresentados, portanto, descrevem a sensibilidade do aprendizado aos hiperparâmetros, mas não permitem afirmar que a política aprendida supera um controlador semafórico convencional.

7. Conclusão

Este trabalho apresentou um ambiente urbano simplificado no qual agentes que navegam e semáforos aprendem por Q-learning tabular, a partir de uma formulação inspirada em estudos sobre controle semafórico e navegação em grade. Foram detalhados os estados, as ações e o modelo de recompensa de cada tipo de aprendiz, a política gulosa obtida a partir das tabelas e os hiperparâmetros envolvidos. Na comparação de execuções, observou-se que **[principais achados e contribuições]**.

Como trabalhos futuros, a primeira etapa é comparar a política aprendida com o controle de tempo fixo e repetir cada configuração com várias sementes, de modo a permitir uma análise estatística. Também é possível incorporar ao agente de navegação o mapa topológico e a exploração guiada de Hafez e Loo [Hafez e Loo 2015]. Por fim, o estado dos semáforos pode ser enriquecido com a ocupação de cada célula dos bra-

ços do cruzamento e comprimido pela análise de componentes principais de Curran et al. [Curran et al. 2015], para comparar o compromisso entre a velocidade de convergência e o desempenho da política em relação ao estado compacto adotado neste trabalho.

Referências

- Curran, W., Brys, T., Taylor, M. E., e Smart, W. D. (2015). Using PCA to efficiently represent state spaces. *arXiv preprint arXiv:1505.00322*. Disponível em <https://arxiv.org/abs/1505.00322>.
- Hafez, M. B. e Loo, C. K. (2015). Topological Q-learning with internally guided exploration for mobile robot navigation. *Neural Computing and Applications*, 26:1939–1954. Disponível em <https://doi.org/10.1007/s00521-015-1861-8>.
- Jin, Z., Zhu, W., e Song, J. (2011). Implementing traffic signal optimal control by multiagent reinforcement learning. Em *2011 International Conference on Computer Science and Network Technology (ICCSNT)*, p. 2578–2582. IEEE.
- Ng, A. Y., Harada, D., e Russell, S. (1999). Policy invariance under reward transformations: theory and application to reward shaping. Em *Proceedings of the 16th International Conference on Machine Learning (ICML)*, p. 278–287.
- Sutton, R. S. e Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 ed.
- Watkins, C. J. C. H. e Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4):279–292.